

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I K Madan Mohan Reddy (192411206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kyreddy Madan

Case study

QUESTIONS:2

Total Marks:100

Annexure A

Case study

1. (Reg. No. 192411206)

a) E-Learning Platform

An educational institution wants to build an e-learning system where instructors upload courses, students enroll, take quizzes, and receive certificates. The system should support scalability and frequent updates.

Question: Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

b) Smart Parking System

A city introduces a smart parking system where users can check parking availability, reserve slots, and make payments through a mobile app. Sensors update real-time availability.

Question: Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data and evolving requirements, and explain why.

a) E-Learning Platform

Introduction

An educational institution plans to develop an E-Learning Platform where instructors can upload courses, students can enroll in courses, take quizzes, track progress, and receive certificates upon successful completion. Since the system must support scalability and frequent updates, a modular and flexible software design approach is essential.

Modular Software Design Using Object-Oriented Techniques

Object-Oriented Programming (OOP) helps in creating a modular, reusable, and maintainable software system. The following OOP concepts can be applied:

1. Class and Object

A class acts as a blueprint, while objects represent real-world entities.

Examples:

- Student Class
- Instructor Class
- Course Class
- Quiz Class
- Certificate Class

This division helps organize the system into independent modules.

2. Encapsulation

Encapsulation hides internal data and allows access only through methods.

Example:

The Student class stores personal details privately and provides methods such as:

- Enroll Course()
- Take Quiz()
- View Certificate()

Benefits:

- Data security

- Easy maintenance
- Reduced complexity

3. Inheritance

Inheritance allows one class to acquire properties of another.

Example:

User

- Student
- Instructor
- Administrator

All users share common attributes such as:

- User ID
- Name
- Login Credentials

This reduces code duplication and improves reusability.

4. Polymorphism

Polymorphism allows the same method to perform different functions.

Example:

Display Dash board ()

- Student Dashboard → Shows enrolled courses.
- Instructor Dashboard → Shows uploaded courses.
- Admin Dashboard → Shows system reports.

Benefits:

- Flexibility
- Extensibility
- Better maintainability

5. Abstraction

Abstraction hides implementation details and shows only essential features.

Example:

Students only see:

- Enroll Course
- Attempt Quiz
- Download Certificate

The internal processing remains hidden.

6. Modular Architecture

The system can be divided into separate modules:

User Management Module

- Registration
- Login
- Profile Management

Course Management Module

- Course Creation
- Course Upload
- Course Update

Quiz Management Module

- Quiz Creation
- Quiz Evaluation
- Score Calculation

Certificate Management Module

- Certificate Generation
- Certificate Download

Reporting Module

- Student Progress
- Performance Analytics

Advantages:

- Easy maintenance
- Independent updates
- Better scalability
- Improved testing

Evaluation of SDLC Models**1. Waterfall Model**

Advantages

- Simple and easy to understand
- Well-documented process

Disadvantages

- Difficult to accommodate changes
- User feedback comes late
- Not suitable for evolving requirements

Suitability

Not recommended because e-learning platforms require continuous updates and feature additions.

2. Spiral Model

Advantages

- Risk analysis at every phase
- Suitable for large projects

Disadvantages

- Expensive
- Complex management

Suitability

Useful for high-risk systems but may be costly for an educational platform.

3. Prototype Model

Advantages

- Early user feedback
- Better requirement understanding

Disadvantages

- Can increase development time
- Frequent modifications

Suitability

Good during initial requirement gathering but not ideal as the primary development model.

4. Agile Model

Advantages

- Supports frequent updates
- Continuous customer feedback
- Incremental development
- Faster delivery of features
- Easy adaptation to changing requirements

Disadvantages

- Requires active stakeholder involvement
- Frequent meetings needed

Suitability

Highly suitable for an E-Learning Platform.

Recommended SDLC Model: Agile Model

The **Agile Model** is the most appropriate SDLC model for the E-Learning Platform because:

1. Requirements may change frequently.
2. New features such as AI tutoring, live classes, and analytics can be added incrementally.
3. Continuous feedback from students and instructors can be incorporated.
4. Supports scalability and modular development.
5. Faster release cycles improve user satisfaction.
6. Easier maintenance and system updates.

Conclusion

The E-Learning Platform can be effectively developed using object-oriented techniques that promote modularity, reusability, and maintainability. Concepts such as encapsulation, inheritance, polymorphism, and abstraction help divide the system into independent modules like user management, course management, quiz management, and certificate generation. This modular structure simplifies testing, debugging, maintenance, and future enhancements.

After evaluating various SDLC models, the Agile Model emerges as the most suitable approach for this project. Unlike traditional models, Agile supports continuous development, regular user feedback, and rapid adaptation to changing educational requirements. Since e-learning systems frequently require updates such as new courses, improved assessment methods, enhanced security features, and integration with emerging technologies, Agile provides the flexibility needed to implement these changes efficiently.

Furthermore, Agile's iterative development process ensures that working versions of the platform are delivered at regular intervals, allowing students, instructors, and administrators to provide feedback throughout the project lifecycle. This leads to higher software quality, better user satisfaction, and reduced development risks.

b) Smart Parking System

Introduction

A Smart Parking System enables users to find available parking spaces, reserve slots, and make online payments through a mobile application. Parking sensors installed in parking areas continuously update the availability status in real time. The system helps reduce traffic congestion, saves time, and improves parking management efficiency.

Applying Object-Oriented System Development Principles

Object-Oriented System Development (OOSD) helps in creating a scalable, maintainable, and reusable Smart Parking System.

1. Class and Object

Classes represent real-world entities, while objects are instances of those classes.

Main Classes:

- User
- Parking Slot
- Parking Area
- Reservation
- Payment
- Sensor
- Mobile App

Example:

- User object → Raghu
- Parking Slot object → Slot A101
- Payment object → Online Transaction

2. Encapsulation

Data and methods are grouped within a class, protecting internal information.

Example:

Parking Slot Class

- Slot ID
- Slot Status

Methods:

- Check Availability()
- Reserve Slot()
- Release Slot()

Benefits:

- Data security
- Better control over access
- Easier maintenance

3. Inheritance

Inheritance reduces code duplication by creating specialized classes.

Example:

User

- Student/User
- Premium User
- Administrator

Common attributes:

- User ID
- Name
- Mobile Number

Specific functions can be added to child classes.

Benefits:

- Code reusability
- Reduced redundancy
- Easier system expansion

4. Polymorphism

The same method can perform different functions depending on the object.

Example:

Make Payment ()

- Credit Card Payment
- Debit Card Payment
- UPI Payment
- Net Banking Payment

Each payment method processes transactions differently.

Benefits:

- Flexibility

- Extensibility
- Improved maintainability

5. Abstraction

Users interact only with necessary functionalities while implementation details remain hidden.

User Functions:

- Check Parking Availability
- Reserve Slot
- Cancel Reservation
- Make Payment

The complex sensor communication and database operations remain hidden.

Benefits:

- Simplified user interaction
- Reduced system complexity

6. Association

Objects communicate with one another.

Example:

- User reserves Parking Slot.
- Reservation is linked with Payment.
- Sensor updates Parking Slot status.

This interaction helps maintain real-time parking information.

Modular Design of Smart Parking System

User Management Module

- Registration
- Login
- Profile Management

Parking Management Module

- Check Availability
- Reserve Slot
- Release Slot

Sensor Management Module

- Detect Vehicle Entry
- Detect Vehicle Exit

Payment Management Module

- Online Payment
- Payment Verification
- Transaction History

Notification Module

- Reservation Confirmation
- Payment Success Alerts
- Parking Expiry Reminders

Evaluation of SDLC Models

1. Waterfall Model

Advantages

- Simple and structured
- Easy documentation

Disadvantages

- Difficult to accommodate requirement changes
- Not suitable for real-time systems

Suitability

Not recommended.

2. Prototype Model

Advantages

- Early user feedback
- Better requirement understanding

Disadvantages

- Frequent redesigns
- Increased development effort

Suitability

Useful during initial stages only.

3. Agile Model

Advantages

- Supports changing requirements
- Frequent software releases

- Continuous customer feedback
- Easy feature enhancement

Disadvantages

- Requires active stakeholder participation

Suitability

Highly suitable for evolving requirements.

4. Spiral Model

Advantages

- Continuous risk analysis
- Iterative development
- Suitable for large and complex systems

Disadvantages

- Higher cost
- Complex management

Suitability

Suitable for large-scale smart city projects.

Recommended Life Cycle Model: Agile Model

The **Agile SDLC Model** is the most appropriate for the Smart Parking System because:

1. Real-time parking requirements may change frequently.
2. New features such as AI-based parking prediction, dynamic pricing, and navigation support can be added incrementally.
3. Continuous feedback from users and city administrators can be incorporated quickly.
4. Sensor integration and mobile app updates can be released in small iterations.
5. Faster deployment and maintenance improve system performance.
6. Supports scalability for expanding parking locations across the city.

Conclusion

The Smart Parking System can be efficiently designed using object-oriented principles such as Encapsulation, Inheritance, Polymorphism, Abstraction, and Association, which promote modularity, reusability, and easy maintenance. The system can be divided into independent modules such as user management, parking management, sensor management, payment processing, and notification services, making it scalable and adaptable to future enhancements.

Among the SDLC models, the Agile Model is the best choice because it supports continuous integration of real-time sensor data, frequent software updates, and evolving user requirements. Agile enables developers to deliver working versions of the system in short iterations, gather feedback from users and city authorities, and quickly implement improvements. This approach ensures that the Smart Parking System remains reliable, efficient, user-friendly, and capable of supporting future smart city initiatives.

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and	Organized and understandable	Limited clarity and structure	Poor clarity; difficult to
		easy to understand	presentation		understand